Informe técnico – Segundo avance

Pipeline ETL: temperatura máxima y radiación solar en Ibagué (2006–2016)

Integrantes: David Casallas, Samuel Sarmiento, Tomas Cardenas
Entrega: Segundo avance, 9 de noviembre de 2026
Repositorio: Proyecto_Pipeline (GitHub)

1. Resumen

Este informe documenta el pipeline ETL construido para integrar dos fuentes del IDEAM (radiación solar horaria y temperatura máxima diaria) en un único dataset diario y medir la relación entre ambas variables en Ibagué entre diciembre de 2006 y septiembre de 2016. El pipeline se ejecuta con un solo comando (`python main.py`), cumple las siete transformaciones exigidas, aplica controles de calidad antes y después de la integración y produce un dataset consolidado de **2.237 días** sin nulos ni fechas duplicadas.

Hallazgo inicial: existe una relación positiva y moderada entre la temperatura máxima y la radiación diaria (Pearson 0,600; Spearman 0,583; $R^2 = 0,36$). Es una asociación, no una prueba de causalidad.

2. Problema y objetivo

Pregunta: ¿qué relación existe entre la temperatura máxima diaria y la radiación solar diaria en Ibagué entre diciembre de 2006 y septiembre de 2016?

Objetivo: construir un pipeline reproducible de extracción, diagnóstico, limpieza, transformación, integración y validación que deje un dataset diario listo para el análisis.

3. Fuentes de datos

Ambas fuentes se descargaron del portal de atención al ciudadano del IDEAM en formato CSV y se guardan sin modificar en `data/raw/`.

Archivo	Estación	Variable	Frecuencia	Unidad	Registros
`Radiacion.csv`	Batallón Rooke (21215180)	Radiación solar global horaria validada	Horaria	Wh/m²	57.020
`Temperatura.csv`	Aeropuerto Perales (21245040)	Temperatura máxima diaria	Diaria	°C	3.531

Las dos variables vienen de estaciones distintas; este supuesto se declara como limitación en la sección 10.

4. Arquitectura del pipeline

El proyecto sigue una estructura modular: cada etapa vive en su propio módulo de `src/` y `main.py` solo orquesta el orden de ejecución.

Orden	Etapa	Módulo	Función principal
1	Extracción	`src/extraction.py`	`cargar_csv`
2	Diagnóstico de calidad	`src/cleaning.py`	`diagnosticar`
3	Limpieza	`src/cleaning.py`	`convertir_fecha`, `renombrar_columnas`, `quitar_duplicados_y_nulos`, `quitar_columnas_constantes`
4	Transformación	`src/transformation.py`	`separar_fecha_hora`, `agregar_radiacion_diaria`, `filtrar_cobertura`
5	Integración	`src/integration.py`	`unir_datasets`
6	Validación y resultados	`src/validation.py`	`validar_integracion`, `calcular_resultados`, `graficar_dispersion`

Flujo de datos: `data/raw/*.csv` → DataFrames en memoria → dataset diario integrado en `data/processed/datos_integrados.csv` → resultados y gráfica en `outputs/`.

Estructura de carpetas:

- `data/raw/`: archivos originales, nunca se modifican.
- `data/processed/`: dataset consolidado.
- `src/`: módulos del pipeline.
- `outputs/`: diagnóstico de calidad, resultados y gráfica de dispersión.
- `docs/`: este informe técnico.
- `main.py`, `requirements.txt`, `README.md`.

5. Diagnóstico de calidad (datos originales)

El diagnóstico se ejecuta antes de cualquier modificación y se guarda en `outputs/diagnostico_calidad.csv`.

Revisión	Radiación	Temperatura
Filas	57.020	3.531
Valores nulos	0	0
Filas duplicadas	0	0
Fechas duplicadas	0	0
Valores negativos	0	0
Tipo de la columna `Fecha`	texto	texto

Revisión	Radiación	Temperatura
Registros esperados en serie continua	86.003 horas	3.584 días
Registros faltantes	28.983 horas	53 días

Problemas encontrados:

- No hay nulos ni duplicados, pero sí hay **fechas faltantes**: la radiación tiene datos en 2.542 días y solo 1.469 de ellos con las 24 horas.
- La cobertura anual de radiación es muy desigual: 2006 tiene 23 días, 2009 solo 17 y 2011 tiene 365.
- `Fecha` viene como texto en ambos archivos.
- Las frecuencias no coinciden (horaria frente a diaria), lo que impide unir directamente.
- Ambos archivos tienen columnas con un único valor (estación, unidad, nivel de aprobación).

6. Transformaciones y justificación técnica

#	Transformación	Módulo	Justificación
1	Convertir `Fecha` de texto a fecha-hora	`cleaning.py`	Permite separar fecha y hora y operar con fechas
2	Renombrar columnas (`radiacion_wh_m2`, `temp_max_c`, …)	`cleaning.py`	Ambos archivos llaman `Valor` a la medición; evita confusión tras el merge
3	Eliminar duplicados y nulos	`cleaning.py`	Evita contar datos dos veces o con valores vacíos (en estos datos dio 0 casos)
4	Eliminar columnas constantes y guardarlas como metadatos	`cleaning.py`	No aportan información; su valor queda documentado
5	Separar fecha y hora (`fecha_dia`, `hora`)	`transformation.py`	La fecha sin hora es la clave de unión
6	Agregar radiación de horaria a diaria (suma y `n_horas`)	`transformation.py`	La temperatura es diaria; ambas variables deben estar al mismo nivel
7	Filtrar días con menos de 20 horas de radiación	`transformation.py`	Un día incompleto suma menos por datos faltantes, no por menos sol

Conservación de los datos originales: `data/raw/` nunca se modifica y cada función trabaja sobre una copia del DataFrame.

Umbral de cobertura (transformación 7)

El mínimo de 20 horas (`MIN_HORAS` en `main.py`) es una decisión del equipo, respaldada ejecutando el pipeline con distintos valores:

Mínimo de horas	Días analizados	Pearson	Spearman
1 (sin filtrar)	2.504	0,522	0,541
12	2.430	0,575	0,563
18	2.304	0,592	0,576

Mínimo de horas	Días analizados	Pearson	Spearman
20 (elegido)	**2.237**	**0,600**	**0,583**
22	2.098	0,608	0,588
24	1.443	0,607	0,591

Sin filtrar, la correlación baja a 0,52 porque los días incompletos meten ruido. Desde 18 horas el resultado se estabiliza cerca de 0,6, y exigir 24 horas descarta unos 800 días sin cambiar la conclusión. Con 20 horas se conserva la mayor cantidad de días posible sin alterar el resultado.

7. Integración

- **Método:** `pandas.merge` con `how="inner"`.
- **Clave de relación:** `fecha_dia` (fecha sin hora).
- **Por qué inner:** solo sirven los días que tienen temperatura y radiación a la vez.
- **Prevención de pérdidas y duplicaciones:** antes del merge se comprueba que `fecha_dia` tenga el mismo tipo en ambos lados; después, que el número de filas coincida con el número de días en común.

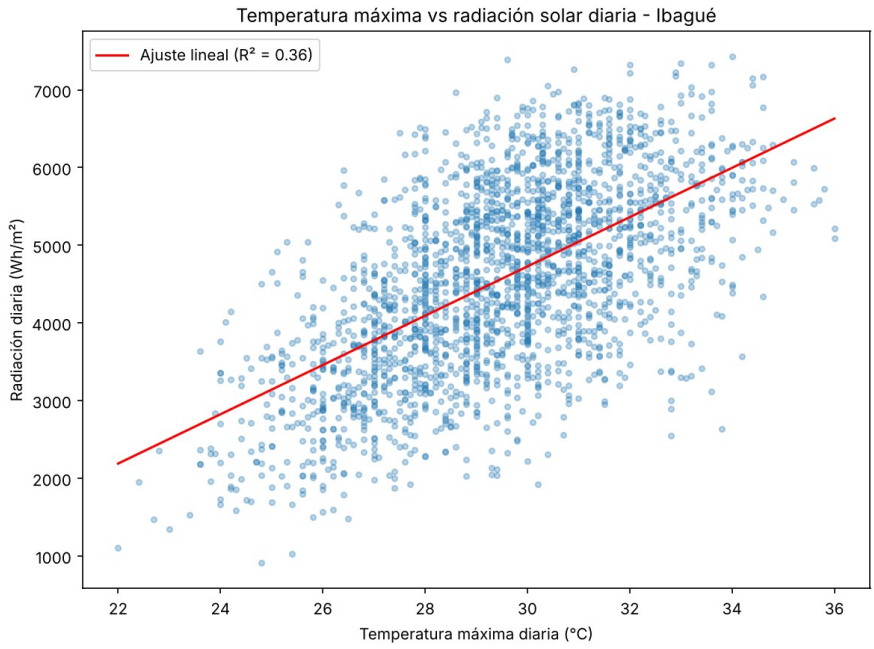
Evolución de los registros: 57.020 horas de radiación → 2.542 días → 2.270 días con al menos 20 horas → **2.237 días** en común con temperatura.

8. Validaciones y resultado preliminar

Las validaciones se ejecutan sobre el dataset integrado y quedan registradas en `outputs/resultados.txt`.

Control	Resultado
Registros del dataset integrado	2.237 (esperados: 2.237)
Fechas duplicadas	0
Nulos en radiación o temperatura	0
Rango de radiación diaria	mínimo 910 Wh/m²
Rango de temperatura máxima	22,0 a 36,0 °C

El dataset consolidado (`data/processed/datos_integrados.csv`) tiene cuatro columnas: `fecha_dia`, `radiacion_diaria`, `n_horas` y `temp_max_c`.



Dispersión de temperatura máxima frente a radiación diaria

9. Hallazgos iniciales

- **Pearson 0,600:** relación positiva, en el límite entre moderada y fuerte según la regla práctica usada.
- **Spearman 0,583:** relación moderada y positiva; al ser parecido a Pearson, la conclusión no depende de valores extremos.
- **Regresión lineal:** por cada +1 °C de temperatura máxima, la radiación diaria es en promedio unos 317 Wh/m² mayor (la media del periodo ronda los 4.560 Wh/m² al día).
- **R^2 = 0,36:** la temperatura máxima se asocia con cerca del 36 % de la variación de la radiación diaria; el resto depende de otros factores, como la nubosidad.

Respuesta a la pregunta del proyecto: entre 2006 y 2016 existe en Ibagué una relación positiva, de moderada a fuerte, entre la temperatura máxima y la radiación solar diaria. Los días más calurosos tienden a ser los de mayor radiación acumulada, pero la dispersión es amplia y la correlación no prueba causalidad.

10. Decisiones técnicas y limitaciones

Decisiones técnicas

- Arquitectura modular con una función por tarea y un único punto de entrada (`main.py`), para que cada etapa pueda probarse y cambiarse de forma aislada.
- Diagnóstico antes de limpiar y guardado en archivo, para poder demostrar el estado inicial de los datos.
- Funciones que trabajan sobre copias, para conservar los datos originales.
- Unión `inner` por `fecha_dia`, porque solo los días con ambas variables sirven para medir la relación.
- Umbral de 20 horas elegido con un análisis de sensibilidad, no de forma arbitraria.

- Gráfica generada con el backend `Agg` de matplotlib, de modo que el pipeline se ejecuta también sin pantalla.
- Rutas construidas con `pathlib` a partir de la ubicación de `main.py`, sin rutas absolutas personales.

Limitaciones

- Las dos variables vienen de estaciones distintas (Batallón Rooke y Aeropuerto Perales).
- Hay años con muy poca radiación registrada (2006 y 2009).
- El umbral de 20 horas es una decisión del equipo; un día con 20 horas suma algo menos que uno con 24.
- Los días consecutivos no son independientes (rachas de días soleados, temporadas secas y lluviosas), por lo que parte de la relación puede deberse a la estacionalidad.
- La correlación y la regresión miden asociación, no causa y efecto.

11. Registro de depuración

Ejemplo del ciclo ejecutar → identificar → corregir → volver a probar, aplicado al repositorio:

Paso	Detalle
Problema detectado	Al ejecutar `python main.py` el programa se detenía con `AttributeError: module 'src.transformation' has no attribute 'separar_fecha_hora'`. Los archivos `src/transformation.py` y `src/validation.py` estaban vacíos (0 bytes), también en el commit del repositorio.
Corrección realizada	Se implementaron `separar_fecha_hora`, `agregar_radiacion_diaria` y `filtrar_cobertura` en `transformation.py`, y `validar_integracion`, `calcular_resultados` y `graficar_dispersion` en `validation.py`, con las firmas que `main.py` ya esperaba.
Resultado de la nueva prueba	El pipeline terminó sin errores: 2.542 días de radiación → 2.270 con al menos 20 horas → 2.237 integrados, con las 4 validaciones en `[OK]`. Las salidas (`datos_integrados.csv`, `resultados.txt`, `diagnostico_calidad.csv`) coinciden con las guardadas previamente.

12. Cómo reproducir

1. Instalar Python 3 y, desde la raíz del proyecto, crear y activar un entorno virtual (`python -m venv venv`).
2. Instalar dependencias: `python -m pip install -r requirements.txt` (pandas, matplotlib, scipy).
3. Verificar que `Radiacion.csv` y `Temperatura.csv` estén en `data/raw/`.
4. Ejecutar `python main.py` (funciona desde cualquier carpeta).

Salidas: `data/processed/datos_integrados.csv`, `outputs/diagnostico_calidad.csv`, `outputs/resultados.txt` y `outputs/dispersion_temp_vs_radiacion.png`.

13. Controles de entrada y manejo de errores

El pipeline valida sus entradas y se detiene con un mensaje claro y código de salida 1; nunca continúa en silencio con datos que no puede interpretar.

Situación	Dónde se controla	Mensaje
Falta un archivo de `data/raw/` o no es `.csv`	`extraction.py`	`No se encontró el archivo: ...`
El CSV está vacío o no se puede leer	`extraction.py`	`No se pudo leer ... como CSV`
Faltan columnas esperadas (`Fecha`, `Valor`, ...)	`extraction.py`	`faltan columnas [...]`
`Valor` contiene texto en vez de números	`extraction.py`	`la columna 'Valor' debe ser numérica`
El dataset integrado queda vacío o un control da `[FALLA]`	`main.py`	`La validación del dataset integrado falló` (no se guarda el dataset)

Estos controles se probaron provocando cada fallo (archivo faltante, columna faltante, texto en `Valor` y fechas sin coincidencia) y en todos los casos el pipeline se detuvo con el mensaje esperado y sin alterar los datos originales.